

Internship Project Journey Report

(Shine Internship Program Cohort 8)

Project: Air Quality Index (AQI) Prediction System
(For Hyderabad, Sindh)

Created By: Abdul Basit

Organization: 10 pearls

Stream lit Dashboard URL: <https://aqi-predictor-ycfkvhequtbkxrxborp6mj.streamlit.app/>

Environment Setup:

I have developed the project on Windows 11 using Python 3.11 (Latest version did not support Hopsworks), managed through a virtual environment. Initial dependency installation revealed two compatibility issues.

The first was a build failure for the twofish package, a Hopsworks dependency, which requires a C compiler absent by default on Windows. This was resolved by installing Microsoft C++ Build Tools with the Desktop Development with C++ workload.

The second was a version conflict between the installed Hopsworks client (4.8.1) and the connected Hopsworks backend (4.7.2), surfaced as a runtime warning. The client was downgraded to match the backend minor version using `pip install hopsworks==4.7.*`.

Data Collection:

I explored different data sources including AQICIN, OpenWeather and Open-Meteo. I chose **Open-Meteo**. Because it provides both the current weather and pollutants data as well as the historical data for my project.

I fetched 1.5 years of hourly historical data (12336 rows) for Hyderabad, Sindh (2025-01-01 to 2026-05-29).

Exploratory Data Analysis (EDA)

Then, I performed EDA to clean and understand data before performing feature engineering. First checked for duplicate rows, missing values and outliers in data (there were none).

Then, check for AQI distribution and it turns out data was almost perfectly balanced (with skew of -0.19), so no transformation was needed.

Then, checked correlation of different data feature with AQI. Dropped the features with insignificant correlation with AQI (*Precipitation* and *Boundary layer Height*).

Then checked how time patterns (*Hour* of day, *Day* of the week, and *Month* of the year), affect AQI. Hour, month and Lag features turned out to be important in predicting future aqi.

Feature Engineering

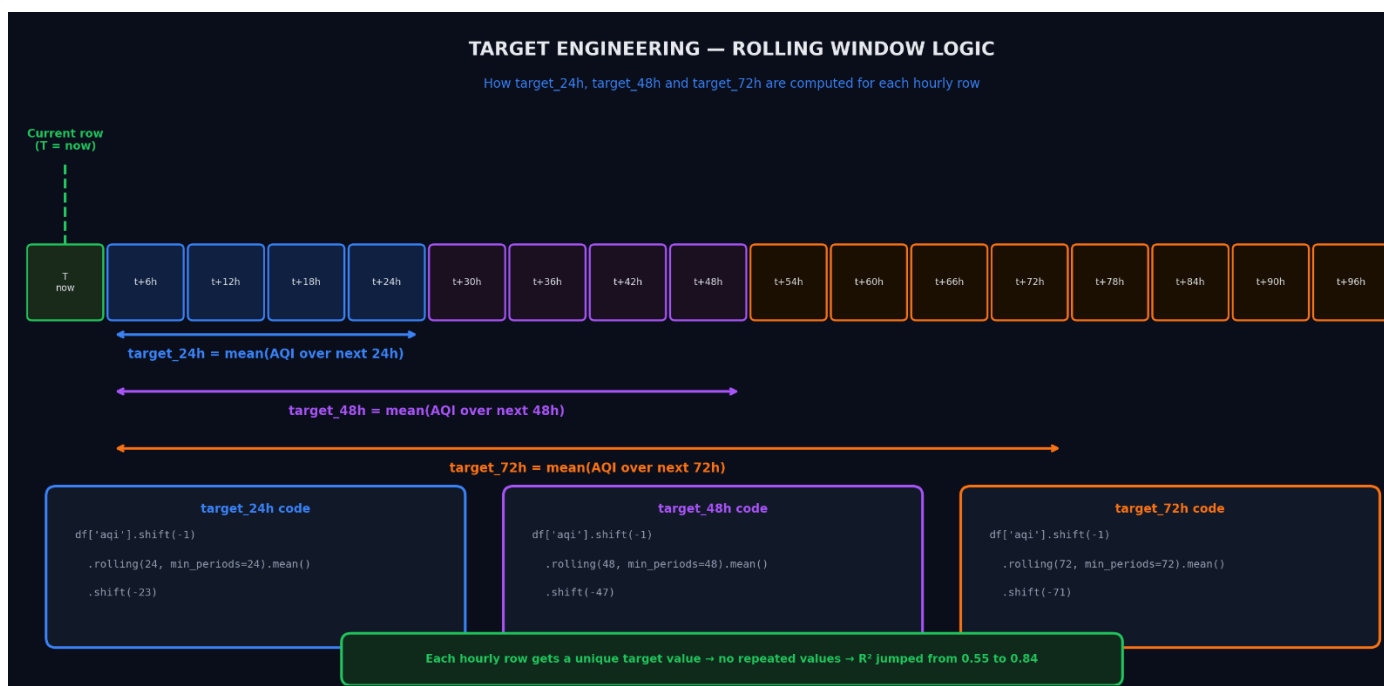
After EDA, I added important features that are helpful in predicting future. Feature Engineering added:

1. Lag Features and Rolling Features for understanding recent trend

2. Time Features (hoursin, hourcos, monthsin, monthcos – circular encoding so that model does not consider 1st and 24th hour far from each other) for affect of hour of the day and month of the year on AQI

3. Targets (target 24h, 48h and 72h):

- Logic: I considered two target logics.
 - Calendar Day Mean AQI Logic: Predict the average AQI for calendar day 1, day 2 and day 3. Model did not perform well on that logic (r^2 of odd 55% for day1), because due to hourly data same mean value was filling the target columns for 24 rows.
 - Next 24h, 48h, 72h Mean AQI Logic: Predict average AQI for next 24h, 48h and 72h. This solved the problem of repeated same target values for 24 rows, because, for each hour mean of next 24, 48 and 72 hours gave different values. Model improved (r^2 rising to 84%).



4. Forecast Weather Features (temp, wind, humidity and cloud) for improving next 48h and 72h predictions.

5. *get_features()* function: This function was included to retrieve the relevant model input features of respective targets.

Feature Store

First, I pushed historical data to Hopsworks Feature Store. But Hopsworks had materialization delay issue. Due to which `fg.read()` showed data from 3 days ago despite successful inserts. So, I switched to **MongoDB Atlas** and the problem got solved.

SO FAR:

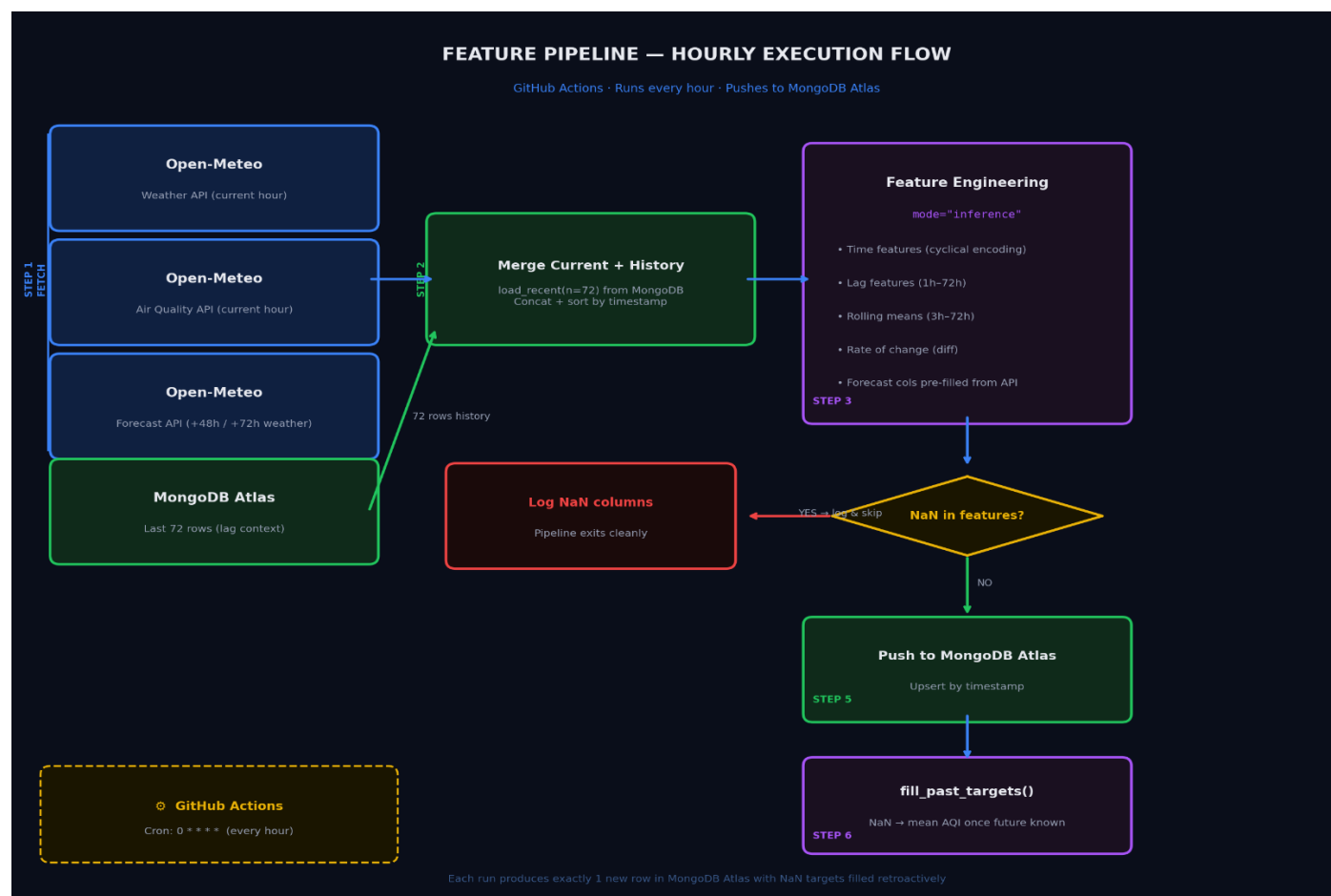
Historical Data Collection -> EDA -> Feature Engineering the Historical Data -> Pushed to MongoDB Atlas

Feature Pipeline

Then I created feature pipeline. It is made to run every hour using Github Actions to store hourly data on MongoDB Atlas. Feature pipeline fetches the current weather and pollutant data from Open-Meteo, and recent data (72 rows) from MongoDB Atlas to calculate lag features and rolling features. Current and recent data is merged, and then feature engineering is performed on the merged data. A row is engineered and is pushed to feature store.

Target values are kept NaNs. I created `fill_past_targets()` function that checks if enough future data is available to fill the target NaNs; once enough future data (14 rows for target 24h, 28 for 48h and 72h) is available, it calculates the averages for the targets and fills them.

14 (not 24) rows because Github Actions does not run the pipeline every hour. It runs every 2 or 3 hours. For 24 rows the function will have to look into 2 days of data to calculate target 24h.



Automation:

I used **Github Actions** to automatically run Feature Pipeline every hour to fetch hourly data.

Model Training:

I trained three models for each forecast horizon — Linear Regression, Random Forest and XGBoost. Initial results were poor (24h $R^2=0.576$, 72h $R^2=0.035$) due to calendar day target logic and no weather forecast features.

Through iterative improvement including:

- changing target logic to next 24h, 48h and 72h
- adding forecast weather features from Open-Meteo (wind, humidity, temp and cloud 48h and 72h)
- adding horizon-specific feature selection (*get_features()* function)

final performance reached

$R^2=0.845$ (24h), $R^2=0.683$ (48h) and $R^2=0.583$ (72h).

Surprisingly, Linear Regression outperformed tree models across all horizons, confirmed by SHAP analysis to reflect genuinely linear relationships in the data rather than a modelling failure.

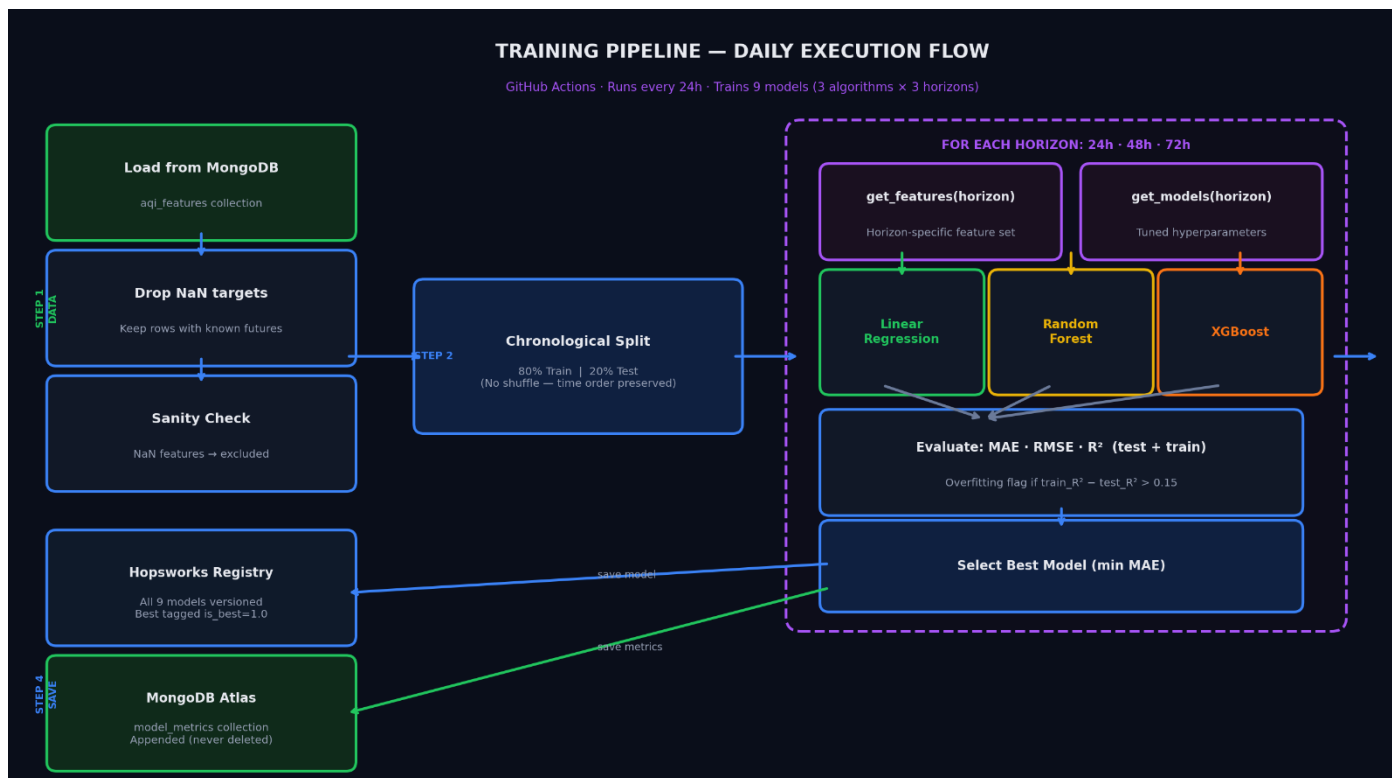
Forecast weather features were the single biggest improvement: 48h R^2 jumped from 0.151 → 0.683 and 72h R^2 jumped from 0.035 → 0.583 - Short-term lag features (aqi_lag_1h, aqi_lag_6h) were irrelevant for 48h/72h targets — removing them improved tree model performance

Model Registry:

I have used **Hopsworks Model Registry** for my models.

Training Pipeline:

Training pipelines runs every 24h via Github Actions. It Fetches data from MongoDB Atlas, splits (80/20) the data in chronological order to avoid any leaks. The Three Models are trained for three target horizons (with their relevant input features) on 80% of the data and tested on 20%. The trained Models are saved to Hopsworks Model Registry. As daily models are trained they are saved with different version. The metrics (MAE, r^2 and RMSE) values of those models are saved to MongoDB Atlas.



SHAP Analysis Summary:

SHAP values were computed for all nine trained models to understand what drives AQI predictions across each forecast horizon.

At the 24-hour horizon, recent AQI history dominates across all three models. The 3-hour and 72-hour rolling AQI means ranked highest, with PM10 and PM25 also contributing strongly. The SHAP summary plots show clear directionality — high rolling AQI values consistently push predictions upward while low values push them down, confirming the predominantly linear relationship that explains why Linear Regression performs best.

At the 48-hour horizon, the 24-hour rolling mean becomes the primary driver, replacing shorter-term signals. Forecast weather features — humidity and temperature at +48h — begin appearing in Linear Regression's top features, validating the decision to include Open-Meteo weather forecasts. Tree models largely ignored these forecast features, continuing to rely on pollutant rolling averages, which partly explains their weaker performance at this range.

At the 72-hour horizon, temperature forecast at +72h emerges as the fourth most important feature for Linear Regression — the highest any forecast feature ranks across all nine models. Tree models show noticeably compressed SHAP values at this horizon, indicating reduced confidence and weaker signal overall.

Across all nine models, the five most consistently important features were `aqi_roll_mean_3h`, `aqi_roll_mean_24h`, `pm25_roll_mean_24h`, `pm25` and `pm10` — appearing in the top 10 of at least six models regardless of algorithm or horizon.

The analysis confirms that AQI prediction is driven primarily by recent pollution levels at short horizons, with forecast weather conditions becoming increasingly important as the prediction window extends.

Dashboard:

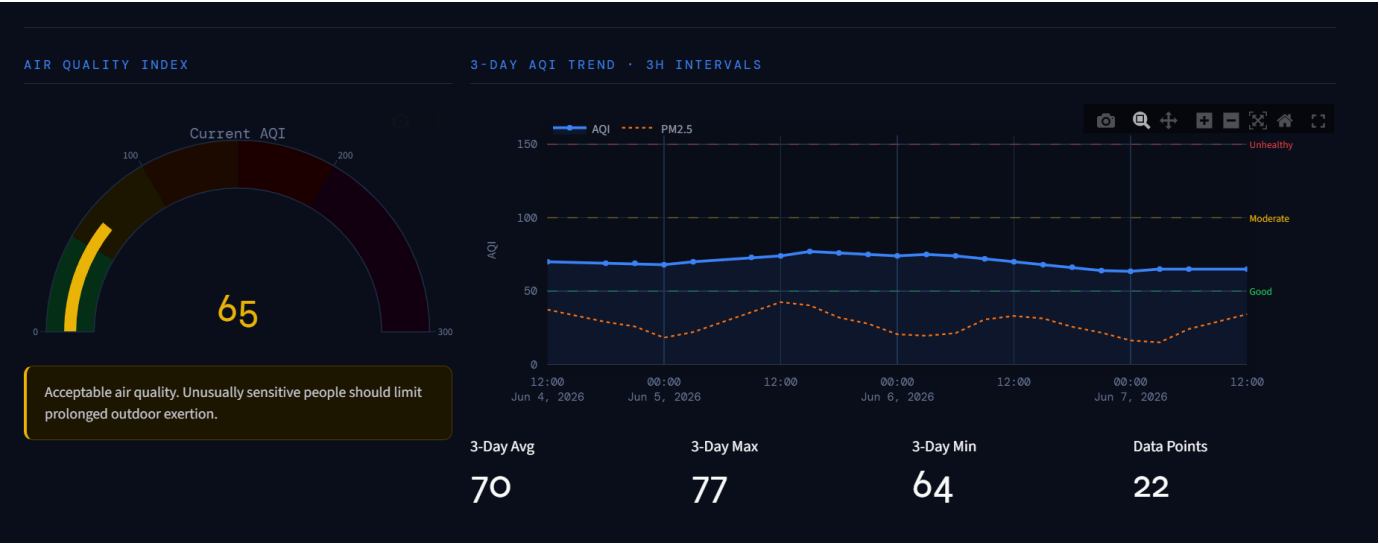
The Dashboard for the system was created using Streamlit and deployed on Streamlit Cloud Community.

The dashboard connects directly to MongoDB Atlas to retrieve the latest hourly features and to Hopsworks to load the best trained model for each forecast horizon dynamically.

The interface displays current AQI with a colour-coded gauge, real-time pollutant readings (PM2.5, PM10, wind speed, humidity and temperature), and a 3-day AQI trend chart resampled to 3-hour intervals to account for GitHub Actions scheduling variability. Three forecast cards present predicted average AQI for the next 24, 48 and 72 hours alongside the model used, its MAE and R² score, and health advisory text tailored to the predicted AQI category. Automated alerts trigger when any forecast exceeds AQI 150. Model performance metrics for the best model per horizon are accessible in an expandable section.

The dashboard refreshes data every 5 minutes and model cache every hour, ensuring predictions always reflect the most recently trained models and latest available features.





Overall System Architecture:

